OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 20

1.Skill20.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_ARGS 64

pid_t shell_pgid;
pid_t foreground_pgid = -1;
```

```
/* Ctrl+C handler */
void handle_sigint(int sig)
{
    (void)sig;

    if (foreground_pgid > 0) {
        kill(-foreground_pgid, SIGINT);
    }
}

/* Ctrl+Z handler */
void handle_sigtstp(int sig)
{
    (void)sig;

    if (foreground_pgid > 0) {
        kill(-foreground_pgid, SIGTSTP);
    }
```

```
}

int main(void)
{
    char command[256];

    /* Get shell PID */
    shell_pgid = getpid();

    /* Ignore terminal control signals in shell */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    /* Create shell process group */
    if (setpgid(shell_pgid, shell_pgid) == -1 &&
        getpgrp() != shell_pgid) {
        perror("setpgid shell");
```

```
}

/* Give terminal to shell */
if (isatty(STDIN_FILENO)) {
   tcsetpgrp(STDIN_FILENO, shell_pgid);
}

/* Register signal handlers */
signal(SIGINT, handle_sigint);
signal(SIGTSTP, handle_sigtstp);

printf("====================================\n");
printf(" OSSP PROGRAM 20\n");
printf(" Process Groups and Terminal Control\n");
printf("====================================\n");

printf("Shell PID  : %d\n", shell_pgid);
printf("Shell PGID : %d\n", getpgrp());
```

```
printf("\nCommands:\n");
printf(" Any Linux command\n");
printf(" exit\n\n");

while (1) {
    printf("shell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strlen(command) == 0)
        continue;

    if (strcmp(command, "exit") == 0)
```

```
    break;

/* Copy command before tokenizing */
char command_copy[256];

strcpy(command_copy, command);

/* Create argument array */
char *args[MAX_ARGS];
int argc = 0;

char *token = strtok(command_copy, " \t");

while (token != NULL && argc < MAX_ARGS - 1) {
    args[argc++] = token;
    token = strtok(NULL, " \t");
}
```

```
args[argc] = NULL;

if (argc == 0)
    continue;

/* Create child process */
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

/* CHILD */
if (pid == 0) {
    /* Create a new process group */
    setpgid(0, 0);
```

```
    /* Restore default signal behavior */
    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);

    /* Execute command */
    execvp(args[0], args);

    perror("execvp");
    _exit(1);
}

/* PARENT */
else {
    /* Put child in its own process group */
    setpgid(pid, pid);
```

```
foreground_pgid = pid;


printf("Child PID  : %d\n", pid);
printf("Child PGID : %d\n", getpgid(pid));
/* Transfer terminal control to child */
if (isatty(STDIN_FILENO)) {
   tcsetpgrp(STDIN_FILENO, foreground_pgid);
}
int status;
/* Wait for child to exit or stop */
if (waitpid(pid, &status, WUNTRACED) == -1) {
   perror("waitpid");
}
/* Return terminal control to shell */
if (isatty(STDIN_FILENO)) {
   tcsetpgrp(STDIN_FILENO, shell_pgid);
}
```

```
        foreground_pgid = -1;


        if (WIFEXITED(status)) {
            printf("Child exited normally.\n");
        }
        else if (WIFSIGNALED(status)) {
            printf("Child terminated by signal %d\n",
                   WTERMSIG(status));
        }
        else if (WIFSTOPPED(status)) {
            printf("Child stopped by signal %d\n",
                   WSTOPSIG(status));
            /* Resume stopped process, as in the source */
            kill(-pid, SIGCONT);
            waitpid(pid, NULL, 0);
        }
    }
}
```

```
    /* Restore terminal to shell */
    if (isatty(STDIN_FILENO)) {
        tcsetpgrp(STDIN_FILENO, shell_pgid);
    }
    printf("\nProgram 20 terminated.\n");
    return 0;
}
```

OUTPUT :

```
sadwika@DESKTOP-8CQAO1L:~$ nano skill20.c
sadwika@DESKTOP-8CQAO1L:~$ gcc -Wall -Wextra -g skill20.c -o skill20
sadwika@DESKTOP-8CQAO1L:~$ ./skill20
====================================
 OSSP PROGRAM 20
 Process Groups and Terminal Control
====================================
Shell PID  : 2622
Shell PGID : 2622

Commands:
 Any Linux command
 exit

shell> pwd
Child PID  : 2629
Child PGID : 2629
/home/sadwika
Child exited normally.
shell> ls
Child PID  : 2630
Child PGID : 2630
GNUstep       exec.c         pipes        skill12.c  skill20    username
Practical     file1.txt      pipes.c      skill13    skill20.c  username.c
Process       fork           process      skill13.c  skill4     wait
S2Commands    fork.c         process.c    skill14    skill4.c   wait.c
Skill         hello          processid    skill14.c  skill5     wait.c.save
a.out         hello.c        processid.c  skill15    skill5.c   wait1
append.txt    input.txt      server       skill15.c  skill6     wait1.c
client        ls_grep        server.c     skill16    skill6.c   week
client.c      ls_grep.c      signals      skill16.c  skill7     week.c
combined.txt  memory         signals.c    skill17    skill7.c   week1
displayPID    memory.c       skill10      skill17.c  skill8     week1.c
displayPID.c  myfifo         skill10.c    skill18    skill8.c
errors.txt    output.txt     skill11      skill18.c  skill9
example.txt   parentchild    skill11.c    skill19    skill9.c
exec          parentchild.c  skill12      skill19.c  trace.txt
Child exited normally.
shell>
```